



SPEED LIMIT SIGN DETECTION AND DRIVER ALERT SYSTEM

Final Report

University of Hull

Supervisor: Baseer Ahmad | Second Marker: Peter Robinson

Module: 600091 – Honours Stage Project | University of Hull

Word count: 15,544

OJONIBE ALEXANDER ABDU

April 2026

Table of Contents

Abstract.....	3
Acknowledgement.....	4
1. Introduction.....	4
1.1 Background to the project	4
1.2 Aims and Objectives	4
1.3 Research question	5
1.4 Report Organization	5
2. Literature Review.....	5
2.1 Detection architecture: the case for single-stage detectors	6
2.2 Confidence threshold calibration	7
2.3 Temporal consistency in video-stream detection	8
2.4 Multi-modal sensor fusion	9
2.5 ADAS alert design and driver behaviour	10
2.6 Simulation environments for ADAS development.....	11
2.7 Summary and identified gaps	11
2.8 Dataset considerations and class imbalance in traffic sign training.....	12
3. Requirements.....	13
3.1 Product requirements	13
3.2 Functional requirements.....	14
3.2.1 Interfaces.....	14
3.2.2 Functional capabilities.....	14
3.2.3 Performance requirements.....	15
3.2.4 Data structures and elements	15
3.2.5 Safety.....	15
3.2.6 Reliability	15
3.2.7 Security and privacy	16
3.2.8 Quality	16
3.2.9 Constraints and Limitations	16
3.3 Design constraints	16
4. Design	17
4.1 System architecture	17

4.2	Module design and responsibilities	17
4.2.1	Frame Processing sequence.....	18
4.3	Decision engine design	19
4.4	Webots car controller	20
4.5	Model training design	21
4.6	Data Logging and evaluation design.....	22
5.	<i>Implementation and testing</i>	<i>22</i>
5.1	Implementation	22
5.1.1	YOLOSpeedDetector	22
5.1.2	DecisionEngine	23
5.1.3	WebotsBridge and WebotsVehicleState	24
5.1.4	AlertManager and Buzzer	24
5.1.5	DataLogger.....	25
5.1.6	Speed car controller.....	25
5.2	Testing.....	26
5.2.1	Unit testing.....	26
5.2.2	Integration testing	26
5.2.3	System testing and keyboard override	27
5.2.4	Formal test plan and results	28
6.	External Concerns	29
6.1	Legal, social and ethical issues	29
6.2	Commercial Issues and exploitation	30
7.	<i>Evaluation and Discussion of Results.....</i>	<i>31</i>
7.1	Detection performance.....	31
7.2	Confidence score analysis	32
7.3	Alert classification.....	33
7.4	Webots Integration reliability	33
7.5	Training baseline metrics	34
7.6	Achievement of Objectives	35
7.7	Limitations and future work	36
7.8	Threats to validity	37
7.9	RAI programme-specific competency	38
8.	<i>Conclusion</i>	<i>38</i>
8.1	Project management.....	38

8.2	Risk Management	39
8.3	General conclusion.....	40
	References	40

Abstract

This report covers the design, implementation and evaluation of a real-time speed limit sign detection and driver alert system built as a final year BSc project. The system uses a YOLOv8 object detection model to identify printed UK speed limit signs from a webcam feed, then cross-references those detections against vehicle speed data and OpenStreetMap road data to decide whether to issue an alert. The main feature that distinguishes this from similar academic work is the ability to tell the difference between a permanent speed limit and a temporary one. That distinction matters in practice because temporary restrictions at roadworks or school zones are frequently missed by existing navigation tools yet carry the same legal weight as any other limit.

The system runs in Python and is split into six modules: camera capture, YOLO-based detection, a decision engine, a Webots simulation bridge, an alert manager, and a data logger. Vehicle speed comes from the Webots robotic simulator over a TCP socket, which removes the need for physical GPS hardware during lab development. A lane-following PID controller drives the simulated car and produces physics-based speed readings that feed directly into the detection pipeline.

Testing covered 1,182 logged frames. The system detected signs in 34.6% of frames, with a mean confidence of 0.917 on successful detections. It correctly flagged 49 overspeed events and 102 temporary sign events. Webots stayed connected in 94.2% of frames. The fully trained model, trained on 100% of the dataset over 100 epochs, produced a [mAP@0.5](#) of 0.989, with precision 0.987 and recall of 0.971 at the 0.60 confidence threshold.

The core research gap this project fills is the absence, in all reviewed literature, of a lightweight system that combines visual sign detection with live GPS speed and map-based limit data. The report argues that the approach is viable as a low-cost driver assistance prototype and contributes to the field of embedded AI for road safety.

Acknowledgement

Thanks Dr Baseer Ahmed for supervision throughout the project and to Peter Robinson for feedback as a second marker. The Roboflow community deserves credit for maintaining an accessible and well-annotated dataset for traffic sign training. The Ultralytics and Webots development teams built the tools this project depends on most heavily. Assistance with code review, and data analysis was provided using Claude (Anthropic, 2026), an AI language model.

1. Introduction

1.1 Background to the project

Speed limit exist to protect road users, but drivers miss them regularly, especially when limits drop temporarily near roadworks, school zones, or incident management sites. The 2023 Department for transport statistics places speeding as a contributing factor in 15% of road fatalities in Great Britian, with temporary restrictions within the most frequently ignored (Department of Transport, 2023). Part of the problem is that standard navigation systems pull from preloaded map databases that can be weeks or months out of date when a new temporary speed restriction goes up.

Advanced driver assistance systems that read speed signs in real time already exist in higher end cars, but they are built into expensive proprietary hardware most drivers cannot access. This project started from a practical question: can you build something similar using cheap off-the-shelf components and open-source software?

The target hardware is aa raspberry pi 5 , which cost around £70.00. A USB webcam and a GPS module handle sensing. The system reads live input, compares detections against current vehicle speed and OSM road data, and fires alerts when a temporary restriction is active or the driver is over the limit. All testing took place using Webots for vehicle physics and printed A4 signs as detection targets.

1.2 Aims and Objectives

The primary aim was to build a working prototype of an AI-powered speed sign detection and alert system capable of identifying temporary restrictions and warning about overspeed conditions. Thew specific objectives were:

- Train YOLOv8 model to recognize UK speed limit signs accurately enough for real-time use.
- Build a modular pipeline that connects camera input, YOLO inference, GPS or simulated speed, and OpenStreetMap data
- Implement logic that distinguishes permanent from temporary speed limits using the gap between a detected sign value and the expected map limit.
- Connect the detection pipeline to the Webots robotics simulator to provide realistic vehicle speed data during development
- Evaluate performance using real logged data and access the results against the original requirements.

1.3 Research question

The central question is whether a low-cost open source system can detect speed limit signs in real-time, distinguish temporary from permanent limits, and issue appropriate alerts on hardware the average driver can afford. A secondary question is whether map-based speed data adds anything over vision-only approach, and whether Webots simulation is an adequate substitute for a real GPS hardware during development

1.4 Report Organization

Section 2 presents the literature review which surveys related academic work on traffic sign recognition, sensor fusion, and ADAS alert design. Section 3 sets out the product and functional requirements. Section 4 describes the system architecture and module design. Section 5 covers the implementation of each component and the testing strategy. Section 6 considers legal, social and ethical issues, as well as commercial exploitation potential. Section 7 presents the evaluation results and discussion, including assessment of the RAI programme-specific competency. Section 8 concludes with project management reflection, risk management review, and general conclusions.

2. Literature Review

This review surveys the academic and technical literature bearing on traffic sign recognition, with a focus on work relevant to the design decisions made in this project. The review is organized around five themes: detection architecture selection, confidence calibration for real-time ADAS, temporal consistency in video-stream detection, multi-modal sensor fusion, and simulation environments for ADAS

development. Where gaps in the literature are identified, these are noted, as they inform the novel contributions of this project.

2.1 Detection architecture: the case for single-stage detectors

The field of automatic traffic sign recognition dates from early 1990s, when researchers began applying colour-based segmentation and template matching to identify regulatory signs in road imagery(de la Escalera et al., 1997)). These early systems were brittle under varied lighting conditions and produced unacceptably high false positive rates in complex scenes. The introduction of convolutional neural network (CNNs) shifted the paradigm substantially. Ciresan et al. (2012) applied a multi-column deep CNN to the German traffic sign recognition Benchmark (GTSRB) and achieved 99.46% accuracy on the classification task, surpassing human-level performance. However, classification assumes the sign has already been cropped from the image; prior detection step remained a bottleneck.

Two-stage detection frameworks, most notably the R-CNN family of architectures (Girshick, 2014; Girshick et al., 2014; Ren et al., 2015), addressed detection and classification together. Li et al. (2016) applied a modified faster R-CNN approach specifically to US speed limit sign detection. Their system used selective search to generate approximately 10,000 region proposals per image, then classified each with a fine-tuned CudaConvNet . They reported an AUC of 89.86% for speed limit signs, a 5.55 percentage point improvement over the previous best result from an ACF detector but noted that throughput on a quad-core CPU was approximately one frame per second. For a real-time driver assistance application, this is categorically insufficient.

Møgelmoose et al. (2015) conducted one of the most methodically thorough benchmarks of traffic sign detection approaches to date, comparing region proposal, sliding window, and CNN-based methods across multiple sign categories on both the US LISA-TS and German GTSDb datasets. Their headline finding for the present project is that speed limit signs are the most difficult category to detect reliably, achieving only 84.31% AUC versus over 96% for categorical signs such as stop signs, yield signs, and warning triangles. The difficulty arises from the fact that all UK and European speed limit signs share identical visual structure (a digit or digits inside a red-bordered white circle), meaning the only distinguishing feature is the numerical content. A detector must therefore be accurate enough at the digit level to discriminate between signs that differ only in whether a given digit is three, a five or a six.

The publication of YOLO (You Only Look Once) by Redmon et al. (2016) provided a different architectural approach. By formulating detection as a single regression

problem over a grid of anchor boxes, and processing the entire image in one forward pass, YOLO achieved real-time throughput of 45 frames per second on a conventional GPU while maintaining competitive accuracy on the PASCAL VOC benchmark. Subsequent versions refined the architecture substantially YOLOv5 (Jocher et al., 2020) introduced CSPDarknet as the backbone and demonstrated strong accuracy-to-speed trade-offs across model size variants from nano to extra-large. YOLOv8, released by Ultralytics in 2023, replaced the coupled detection head with a decoupled anchor-free design, improving mean Average Precision at equivalent inference speeds (Ultralytics, 2023).

Zhang et al. (2024) validated YOLOv8 specifically for traffic sign detection in their CR-YOLOv8 system, which achieved 67.4% mAP across 45 traffic sign classes in real-time. Critically, they demonstrated that YOLOv8's Feature Pyramid Network (FPN) handles multi-scale object detection well, detecting both near and distant signs reliably in the same forward pass. This property is important for vehicular use, where a sign at 50 metres will occupy a very different number of pixels than the same sign at 5 metres.

On the basis of this literature, YOLOv8 nano (YOLOv8n) was selected as the detection architecture for this project. The nano variant is the smallest and fastest in the YOLOv8 family. Trik et al. (2023) demonstrated that nano-scale YOLO models are deployable on a Raspberry pi 4 at 20 – 30 fps, suggesting the Raspberry pi 5 target hardware is achievable with further optimisation. The single-stage architecture eliminates the proposal generation step that makes two-stage detectors too slow for real-time use, and the FPN backbone handles the scale variation inherent in a moving vehicle sensing scenario.

2.2 Confidence threshold calibration

Academic benchmarks almost universally report detection performance at the framework default confidence threshold, which for YOLO is 0.25. At this setting, recall is maximised at the cost of precision: many detections are reported, but a significant fraction are false positives.

For an ADAS alert system, this trade-off is the wrong way around. Gomes et al. (2017), in a study of embedded traffic sign recognition on Beaglebone and Raspberry Pi hardware, noted explicitly that false positive alerts in driver assistance systems carry a significant cost beyond a simple recall reduction. Spurious warning erode driver trust in the system, can cause dangerous overreaction, and lead drivers to disable or ignore the system entirely. Their recommendation was to accept substantially reduced recall in exchange for high precision, using a conservation confidence threshold.

This recommendation is supported by the broader ADAS usability literature. Lee et al.(2004) reviewed the effects of in-vehicle information systems alerts on driver behaviour and found that poorly calibrated or excessive alerts produced a counter-productive distraction effect, increasing rather than decreasing accident risk. Specifically, they identified that drivers began to tune out systems that alerted frequently without consequence, reducing their responsiveness to genuine warnings. These findings directly motivated the selection of a confidence threshold, per-frame recall is reduced, but each detection that is reported can be treated as reliable. The multi-frame confirmation mechanism, described in section 4.3, further compensated for the recall reduction by accumulating evidence across consecutive frames.

2.3 Temporal consistency in video-stream detection

A structural gap exists in the traffic sign recognition literature between the evaluation methodology used in academic benchmarks and the deployment reality of a video-stream-based system. The GTSRB, TT100K , and LISA-TS benchmarks all evaluate detection on still images or independently sampled video frames, reporting per-frame metrics such as precision, recall, and mAP. This methodology does not capture the behaviour of a system processing a continuous video stream, where the same physical sign will appear in many consecutive frames as the vehicle approaches it, and where individual frames may suffer from motion blur, partial occlusion at a signpost, or unusual viewing angles at the moment of capture.

Li et al. (2016) acknowledged some of this implicitly through the use of hard negative mining – retraining the classifier on images it had misclassified – but this addresses the training distribution rather than the inference-time temporal structure of a video stream. Triki et al. (2023) tested their lightweight CNN system on video and reported frames-per-second metrics, but their evaluation remained per-frame accuracy rather than exploiting temporal consistency across the stream.

The consequence of this gap is that systems optimised for per-frame accuracy benchmarks will tend to accept transient false positives: a partial sign visible for a single frame, a sign on the opposite carriageway briefly in view, or a reflection that momentarily resembles a speed sign. In a deployed alert system, each such transient false positive would trigger an alert, contributing to the false positive rate that, as discussed in section 2.2, degrades driver trust.

The multi-frame temporal confirmation mechanism implemented in this project – requiring that the same speed value be detected in N consecutive frames before confirmation addresses this gap directly. The approach is not novel in principle; similar

idea appears in pedestrian detection systems (Dalal and Triggs, 2005) where temporal coherence is used to reduce false positive rates. However, its application to speed sign recognition in a video-stream ADAS context does not appear to have been systematically evaluated in the published literature reviewed for this project. The default parameter of $N=3$ was chosen as a balance between false positive suppression (higher N reduces false positives more aggressively) and detection latency (lower N produces confirmation faster). At a processing rate of 7.7fps, $N=3$ corresponds to a confirmation latency of approximately 0.4 seconds, which is acceptable given that speed limit signs typically remain in the camera field of view for several seconds as a vehicle approaches.

2.4 Multi-modal sensor fusion

The most significant gap identified across the academic literature surveyed is the near-total absence of systems that integrate map-based road speed data with visual sign detection. Every paper reviewed treats traffic sign recognition as a pure computer vision problem: a camera frame is analysed, a sign is or is not detected, and the result is reported. No reviewed system asks whether the detected value is contextually plausible given the vehicle's location, or whether a deviation from the expected map limit might indicate a temporary restriction rather than a permanent one.

This gap has practical consequences. A detection system operating on vision alone cannot distinguish between a permanent 30 mph sign in a residential street and a temporary 30 mph sign erected at a motorway roadworks. The alert it would produce is identical in both cases. However, from the perspective of driver assistance, these two situations are qualitatively different: the temporary restriction is the more safety critical case, since drivers are less likely to have anticipated it, and the existing literature on human factors in road safety supports stronger alerting for unexpected restriction (Stanton and Pinto, 2000).

OpenStreetMap (OSM) provides freely available road network data including speed limit tags through the Overpass API (OpenStreetMap, 2024). The maxspeed tag on highway ways records the statutory speed limit for that road segment. In the implementation described in this report, the OSMMAPClient module translates the Webots simulation's pos_x coordinate into a corresponding real UK road coordinate using a zone table, which maps ranges of pos_x values to representative UK road locations with known speed limits (for example, a residential street in Hull at 30 mph, the A63 dual carriageway at 50 mph, and the M62 motorway at 70 mph). The map speed limit is read directly from the zone table entry rather than from a live Overpass API query, making the map integration reliable in simulation without requiring a GPS fix or network

connectivity. A detected sign value that is significantly lower than this map value – 10 mph or more in the implementation described in this report – is classified as a temporary restriction. A detected sign that corresponds to or is within normal tolerance of the map value is classified as permanent.

Veres et al. (2011) argued, in a survey of autonomous vehicle sensor architectures, that sensor fusion combining multiple independent data streams is a prerequisite for robust real-world ADAS performance. Their analysis showed that any single modality – camera, radar, GPS, map – has specific failure modes that can be mitigated by redundant data from other sources. The fusion architecture implemented in this project is comparatively simple – a threshold comparison rather than a probabilistic fusion model – but it implements the same principle: camera-based sign detection is contextualised and validated by map data, and vehicle speed is independently monitored to assess the driver's response.

GPS speed smoothing is a related requirement. Raw GPS speed measurements contain noise from satellite geometry variation, multipath effects in urban canyons, and receiver quantisation. Gomes et al. (2017) noted this issue in their embedded ADAS work and recommended exponential moving average (EMA) filtering for speed data. An EMA with parameter α assigns weight α to the most recent measurement and weight $(1-\alpha)$ to the running average. The choice of $\alpha = 0.25$, used in this project, corresponds to an effective averaging window of 4 samples – sufficient to smooth high-frequency noise while remaining responsive to genuine speed changes.

2.5 ADAS alert design and driver behaviour

The engineering of the alert subsystem was informed by human factors research on driver behaviour in response to in-vehicle alerts. Lee et al. (2004) identified three key design principles for ADAS alert systems: alerts should fire only after a condition has been sustained for a minimum duration (to avoid transient triggers); repeated alerts for the same ongoing event should be rate-limited (to avoid habituation); and alert modality and timing should minimise attentional demand at safety-critical moments. These principles translate directly into the alert design parameters used in this project: a one-second sustained overspeed threshold before the alert fires, a five-second cooldown between successive alerts of the same type, and a two-mph tolerance margin to avoid alerting on minor speed fluctuations close to the limit.

Stanton and Pinto (2000) specifically examined driver responses to speed limit systems and found that drivers respond more strongly to unexpected limit changes than to

expected ones. This supports the decision to distinguish between temporary and permanent sign alerts and to treat temporary signs as the higher-priority category, with an audible buzzer alert in addition to the visual console warning.

2.6 Simulation environments for ADAS development

The use of simulation for ADAS development avoids safety, legal, and practical obstacles associated with real-road testing. Webots, originally developed by Michel (2004) at EPFL and now maintained by Cyberbotics, is a widely used open-source robotics simulation platform. It provides a physics engine capable of modelling vehicle dynamics, simulated sensors including GPS, cameras, LiDAR, and a controller API available in Python, C, C++, Java and MATLAB. Its use in academic ADAS research is well-established.

Dosovitskiy et al. (2017) developed CARLA, a high-fidelity simulation environment specifically designed for autonomous driving research, built on the Unreal Engine 4 rendering pipeline. CARLA provides photorealistic urban environments and supports a wide range of sensor configurations. However, its computational requirements are substantially higher than Webots, and its complexity makes it less appropriate for a single-author project with a tight development timeline. Webots provides sufficient physics fidelity for the specific need in this project: generating realistic GPS-equivalent speed data for vehicle navigating a simple road environment.

The integration architecture used here – broadcasting vehicle state over a TCP socket at 10 Hz from the Webots controller to the python detection process – is a design pattern that maps cleanly onto real deployment scenario, where a GPS module would transmit NMEA sentences over a serial interface. The DecisionEngine, AlertManager and DataLogger components are entirely agnostic to whether the speed data they receive comes from Webots or from a real GPS receiver, which means the transition from simulated to hardware-based speed input requires only a change of the data source, not a redesign of the logic.

2.7 Summary and identified gaps

The literature review identifies five observations that collectively frame the contribution of this project. First, YOLOv8 nano is an appropriate architecture for real-time speed sign detection on Raspberry pi 5 hardware, supported by published benchmark results from Zhang et al. (2024) and Triki et al. (2023). Second, a confidence threshold well above the YOLO default is necessary for an alert system, supported by Gomes et al. (2017) and Lee et al. (2004). Third, temporal filtering to exploit video-stream continuity is underexplored in the traffic sign recognition literature, representing an engineering

gap addressed by the multi-frame confirmation mechanism. Fourth, multi-modal fusion of visual detection with GPS and map data is essentially absent from low-cost published systems, representing the primary novel contribution of this project. Fifth, Webots provides a suitable simulation environment for development and testing at the prototype stage

2.8 Dataset considerations and class imbalance in traffic sign training

The composition and balance of the training dataset exerts a strong influence on the behaviour of a learned detection model, and this issue is particularly acute for speed sign recognition. Unlike datasets for general object detection, where several categories each contribute ample sample counts, speed sign datasets are inherently skewed: common limits appear frequently within real-world imagery while rare limits are captured infrequently. Understanding this disproportion is a prerequisite for interpreting the model's per-class behaviour and for reaching informed decisions about threshold calibration and augmentation strategy.

Stallkamp et al. (2012) introduced the German Traffic Sign Recognition Benchmark (GTSRB), which became the primary evaluation benchmark for a decade of traffic sign recognition research. They found that the 43-class dataset had highly unequal sample counts, ranging from over 2,000 training examples for priority road signs down to under 250 for some rare prohibitory signs. Classification accuracy dropped substantially for underrepresented classes even when overall accuracy was high, and they recommended that any system evaluation report per-class performance rather than only the aggregate mean. That recommendation is followed in Section 7.5 of this report. The dataset used in this project, sourced from Roboflow (Roboflow, 2026), shows a similar imbalance. The 13 speed limit classes range from well-represented UK statutory limits (30 mph and 40 mph) to essentially absent European motorway limits (110, 120, and 130 mph). This has two practical consequences. First, the model learns a weaker representation of rare classes because it has fewer examples from which to generalise. In the DEBUG training run, speed limits 110, 120, and 130 mph produced zero confident predictions at test time, which is precisely what the training set composition would predict. Second, the model may be biased towards common classes when presented with an ambiguous input: a blurred or partially occluded sign may be classified as 30 mph rather than 50 mph simply because 30 mph appears far more often in training.

Shrivastava et al. (2016), in their work on hard example mining for object detection, showed that training on a class-balanced subset consistently outperformed training on the full imbalanced dataset for minority classes, even when the total training volume was reduced. Applied here, that finding suggests oversampling or augmenting the rarer UK speed limit classes (50, 60, and 70 mph) would be more effective than simply

increasing the total dataset size uniformly. Synthetic sign generation, where digit values are rendered onto a standardised red-circle template at varying scales and lighting conditions, is one practical avenue for addressing this gap without the labour cost of collecting additional real-world imagery.

The horizontal and vertical flip augmentation settings in the training configuration were both disabled ($\text{fliplr}=0.0$, $\text{flipud}=0.0$) for this reason: mirrored speed signs are not valid sign instances, and including them would add noisy class-imbalanced samples rather than useful diversity. Rotation, translation, scale, and colour augmentations were retained because they produce valid sign instances at different viewing conditions, adding genuine diversity within each class.

The class imbalance issue also bears on confidence threshold selection. In an imbalanced model, the confidence scores for majority classes tend to be calibrated higher than those for minority classes, because the model has learned a stronger feature representation for majority classes. A single global threshold of 0.60 therefore applies a relatively stricter standard to minority classes than to majority classes. A per-class threshold calibration, where each class has its threshold set from its own precision-recall curve, would provide more balanced performance across classes. This is identified as a priority for the full-training model development phase.

3. Requirements

3.1 Product requirements

The system is designed as a proof-of-concept prototype for a low-cost add-on driver assistance device. The intended user is a driver of a vehicle not equipped with a factory ADAS system. The system monitors forward-facing camera feed, infers the current speed limit from detected speed signs and map data, monitors the vehicle's speed via GPS or simulation, and issues alerts when the inferred limit is lower than expected (suggesting a temporary restriction) or when the vehicle is exceeding the confirmed limit.

The target deployment hardware is a Raspberry Pi 5 with 4 GB or more of RAM, connected to a USB webcam and a serial GPS module. For the purpose of deployment and the evaluation described in this report, vehicle speed is provided by a Webots simulation rather than a physical GPS module.

The following product requirements were identified and agreed with the project supervisor:

- PR1: The system shall detect UK speed limit signs with value of 20, 30, 40, 50, 60 and 70 mph from live webcam input
- PR2: The system shall determine whether the vehicle is exceeding the confirmed speed limit and issue an alert if it is.
- PR3: The system shall distinguish between permanent and temporary speed restrictions and issue a distinct alert for temporary restrictions
- PR4: The system shall operate at a frame rate sufficient for real-time use, defined as a minimum of 10 fps on the target hardware
- PR5: The system shall function without an internet connection for the core detection pipeline; the OSM map query shall be optional and shall degrade gracefully if unavailable
- PR6: The system shall connect to a Webots simulation to receive vehicle speed and position data, with graceful fallback to a configurable default speed if the simulation is not running

3.2 Functional requirements

3.2.1 Interfaces

The system uses three external data sources. First, it gets video frames from a USB webcam through the OpenCV VideoCapture API, with a default resolution of 640 x 480 pixels. Second, it connects to the Webots simulation controller over a TCP socket to receive vehicle state data in JSON format at 10 Hz. Third, the OSMMAPClient module supplies road speed limit information for the vehicle's current position by converting the Webots pos_x coordinate into a zone-based speed limit. In simulation, this lookup uses an in-memory zone table, so it works even without internet access. The user interface shows the annotated camera feed in real time in an OpenCV window, and also prints alert messages and status updates to the console.

3.2.2 Functional capabilities

The system needs to capture video frames from a USB webcam and run YOLOv8 on each frame to detect speed limit signs. It should extract speed values from the detected class labels and use a configurable multi-frame buffer to make detections more stable. The system must connect to Webots over TCP, read vehicle state data in JSON format, and use the OSMMAPClient to find the road speed limit for the vehicle's current position. This client automatically maps Webots pos_x ranges to speed limits as the vehicle moves. The system should classify confirmed detections as normal, temporary, or overspeed, based on the map speed and the vehicle's speed. It must provide console and audible alerts with a configurable cooldown, show detection and

status information on the live video, and log all per-frame data to a CSV file for later analysis.

3.2.3 Performance requirements

The system shall process camera frames at a minimum of 10 frames per second on the target Raspberry Pi 5 hardware. The multi-frame confirmation mechanism shall introduce a maximum addition detection latency of 0.5 seconds under normal operating conditions. The Webots bridge shall receive speed updates at no less than 5Hz to maintain speed accuracy for overspeed detection. The alert manger shall respond to a confirm overspeed or temporary sign event within 0.1 second of the event being raised by the decision engine.

3.2.4 Data structures and elements

Each processed frame shall generate a loh entry containing eleven fields:

Timestamp(seconds since session start, float), vehicle_speed_mph(float), map_speed_mph(float), pos_x (float, metres), pos_z (float, metres), webots_connected (integer 0 or 1), detected_speed(integer mph, 0 if no detection), confirmed_speed(integer mph, 0 if no confirmed detection), confidence (float 0.0 to 1.0), is_overspeed (integer 0 or 1), and is_temporary(integer 0 or 1).

Vehicle state receiv_mph, speed_kph, pos_x, pos_z and ts fields. The internal WebotsVehicleState object shall cache the most recently received values and expose them via thread-safe accessors.

3.2.5 Safety

All testing shall be conducted in a controlled indoor laboratory environment. No physical vehicles shall be used in any testing activity. The system shall not interface with any vehicle control system and shall have no capacity to affect braking, steering, throttle, or any other vehicle actuation. The system is an advisory tool only. This requirement aligns with the University of Hull ethical guidelines for student projects and ensures that any detection error or system failures have no physical consequences. The system shall not store any video footage or personally identifiable data.

3.2.6 Reliability

The system shall handle the loss of the Webots TCP connection without crashing, falling back to a configurable default vehicle speed. It shall handle camera frames in which no speed sign is detected without generating spurious alerts. The alert manager shall implement cooldown logic to suppress repeated alerts for a single ongoing event.

The system shall handle malformed or incomplete JSON packets from the Weots bridge without data corruption.

3.2.7 Security and privacy

GPS coordinates used for OSM queries shall not be logged or transmitted beyond the Overpass API request. All OSM queries shall be made over HTTPS. The system shall not collect or store any personal data. Use of OpenStreetMap data is consistent with the Open Database Licence (ODbL) (OpenStreetMap, 2022).

3.2.8 Quality

The software shall be structured as independent modules with clear interfaces, enabling individual components to be tested in isolation. All configurable parameters shall be centralised on a single Config dataclass. Module interfaces shall be documented at the class level. The codebase shall be version controlled using Git.

3.2.9 Constraints and Limitations

The system is constrained to detecting UK speed limit classes (20, 30, 40, 50, 60, and 70 mph). European motorway speed values (110, 120, 130 mph) were present in the training dataset but are poorly represented and are not required by the UK deployment context. The OSM map speed functionality is exercised during evaluation via a zone table in `map_osm.py`, which maps Webots `pos_x` position ranges to speed limit values without requiring a live internet connection or GPS fix; the map speed therefore updates automatically as the simulated vehicle moves. The system's detection accuracy is limited by the training dataset and the fully trained model (`speed_sign_full3`) used in evaluation, achieving a [mAP@0.5](#) of 0.989.

3.3 Design constraints

The project had to be completed within a single academic year using entirely free and open-source tools. All software dependencies must be freely distributable. The implementation language was constrained to Python by Webots controller API and the availability of the Ultralytics YOLO and OpenCV libraries. Model training was performed on a MacBook Pro with Apple M1 GPU acceleration, as this was the primary development machine available. Inference was subsequently tested on the same hardware as a proxy for Raspberry Pi performance; the YOLOv8n model is known to run at comparable speeds on ARM-based hardware.

4. Design

4.1 System architecture

The system is divided into two distinct processes that communicate over local TCP socket connection. The first process is the Webots simulation, which runs the car controller (`speed_car_controller.py`) within the Webots runtime environment. The second process is the main Python detection pipeline (`main_webots.py`), which runs independently and connects to the Webots process as a TCP client. This two-process architecture was chosen for three reasons. Webots requires its own. Runtime environment and embeds Python via its controller API in a way that is not compatible with running additional python processes within the same interpreter. The separation allows each side to be started, stopped and debugged independently, which proved valuable during development. Finally, the architecture mirrors a real deployment scenario, where a GPS receiver hardware module would communicate with a host computer over a serial interface – the Webots TCP socket plays the role of that serial interface.

The Webots simulation runs a BmwX5 vehicle model on a road track with lane markings. The vehicle's GPS sensor records speed in metres per second and works coordinates at each simulation timestep. The car controller reads these values, converts them to common units and broadcasts them as newline-delimited JSON packets to any connected TCP client at 10Hz. The detection pipeline receives these packets through a background thread (`WebotsBridge`), which updates a shared `WebotsVehicleState` object protected by a `threading.Lock`. The main loop reads from this state object on each iteration, processes a webcam frame through the `YoloSpeedDetector`, passes the result to the `DecisionEngine`, manages alerts via the `AlertManager`, and writes a log entry via the `DataLogger`.

4.2 Module design and responsibilities

The python detection pipeline is structured as six modules, each with a single clearly define responsibility. This design was driven by the principle of separation of concerns: each module can be developed and tested independently, and changes to one module do not propagate unpredictably into others. Table 1 lists the modules, their source files, and their responsibilities

Module	File	Primary Responsibility
YoloSpeedDetector	detector.py	YOLOv8 model loading, inference, label parsing, bounding box filtering
DecisionEngine	decision.py	Multi-frame confirmation buffer, temporary/overspeed classification, event generation
WebotsVehicleState	main_webots.py	Thread-safe cache of the most recent vehicle state from Webots
WebotsBridge	main_webots.py	Background TCP client thread; parses Webots JSON packets and updates vehicle state
AlertManager / Buzzer	main_webots.py	Cooldown-gated console and audio alerts for TEMPORARY and OVERSPEED events
DataLogger	main_webots.py	Per-frame in-memory accumulation and CSV export on shutdown
SpeedCarController	speed_car_controller.py	Webots car controller: PID lane following, obstacle avoidance, GPS broadcast server

4.2.1 Frame Processing sequence

Understanding this sequence is useful to reason about the system's latency and for identifying which components are on the critical path between frame capture and alert output.

The sequence begins when the main loop calls `camera.read()` on the OpenCV `VideoCapture` object. This is a blocking call: the main thread waits until the next frame is available from the camera driver. At 7.7 fps the wait is approximately 130 milliseconds. The frame is then passed to `YoloSpeedDetector.detect()`, which runs the full YOLOv8 inference pipeline including pre-processing (resize to 640x640, pixel normalisation), the forward pass through the network, and post-processing (decode anchor-free predictions, apply NMS, filter by confidence threshold and minimum bounding box area). On the development hardware this step takes approximately 50 to 80 milliseconds and is the dominant cost in the processing loop.

The top detection (or `None`) is then passed to `DecisionEngine.update()`, along with the current vehicle speed in metres per second and the current map speed limit in mph. Both values are read from `WebotsVehicleState.get_snapshot()`, which acquires the threading lock, copies the current state dictionary, and releases the lock. In normal operation the lock is not contested: the `WebotsBridge` background thread holds it only

for the duration of writing four float values, which takes nanoseconds. The DecisionEngine update call is synchronous and completes in microseconds, returning either None or an event dictionary.

If an event is returned, the main loop calls AlertManager.alert() for each applicable alert type (TEMPORARY or OVERSPEED). The AlertManager checks its per-type cooldown dictionary and, if the cooldown has elapsed, writes a colour-coded message to the console and, for TEMPORARY events, calls Buzzer.buzz(). The Buzzer launches the system sound process asynchronously using os.system with a trailing ampersand, so audio playback does not block the main loop. DataLogger.log() is called unconditionally for every frame, appending a dictionary to the in-memory list in microseconds with no I/O cost. Finally, if show_video is enabled, the frame is annotated and passed to cv2.imshow(), followed by cv2.waitKey(1) to process keyboard events.

The WebotsBridge thread runs concurrently with the main loop and is not part of the per-frame sequence because it operates independently. It blocks on socket.recv() until data arrives, parses the JSON packet, and updates WebotsVehicleState under lock. This update occurs every 100 milliseconds from the Webots side, which is faster than the main loop's frame period, so the vehicle state read by the main loop is always current.

4.3 Decision engine design

The DecisionEngine is the intellectually central component of the system. It receives three inputs on each processed frame: the highest-confidence detected speed value from the YoloSpeedDetector (or None if no detection was made), the current vehicle speed in metres per second from the Webots bridge (or None if disconnected), and the current road speed limit in mph from the OSMMAPClient zone table, which updates automatically as the vehicle's pos_x position crosses zone boundaries. It produces an event dictionary when a confirmed detection is available, or None otherwise.

Detection confirmation uses a collections.deque of maximum length confirm_frames (default 3). Each frame in which a detection is made appends the detected speed to the deque. A frame in which no detection is made clears the deque entirely and resets the current_sign_mph state. This immediate-clear-on-miss behaviour is significant: it ensures that a sign that briefly leaves the camera field of view (because the car has turned, or the camera is blocked) does not produce a stale confirmed detection that would generate incorrect alerts on subsequent frames. A detection is confirmed only when the deque has accumulated exactly confirm_frames entries and all of them are identical.

The temporary sign classification compares the confirmed speed value against the map speed limit. If the confirmed value is less than or equal to (map_speed_mph – temporary_gap_mph), the event is flagged as temporary. The default

temporary_gap_mph of 10 mph was chosen on the basis that normal OSM data inaccuracy and map update lag would rarely produce a gap between the actual road limit and the map-recorded limit of more than 5 mph, whereas a genuine temporary restriction (roadworks, school zone, incident management) would typically produce a gap of 15–30 mph in a typical UK context. The 10 mph threshold provides a conservative margin between these two cases.

Overspeed detection uses a time-gated approach. When the vehicle speed exceeds the confirmed sign speed by more than overspeed_tolerance_mph (default 2.0 mph), a timestamp is recorded as overspeed_start_ts. The is_overspeed flag is set to True only when the current time exceeds overspeed_start_ts by at least overspeed_hold_seconds (default 1.0 seconds), meaning the vehicle must have been above the limit for a sustained period. If the vehicle speed drops back within tolerance at any point, overspeed_start_ts is cleared. This debouncing prevents the engine from flagging brief acceleration events, consistent with Lee et al.'s (2004) recommendation against alerts triggered by transient speed fluctuations.

4.4 Webots car controller

The webots car controller implements four main capabilities: automated lane following using camera-based yellow line tracking, obstacle detection and avoidance using the SICK LiDAR scanner, keyboard-driven manual control for speed and steering adjustments and GPS state broadcasting over TCP.

The lane-following algorithm checks each camera frame for pixels with BGR colour values close to a set yellow reference (B=95, G=187, R=203, tolerance=30). It calculates the mean x-position of these yellow pixels by averaging their x-coordinates. This mean is then adjusted to a range between -0.5 and +0.5, and multiplied by the camera's field of view in radians to get an angular error signal. A PID controller with gains $K_p=0.25$, $K_i=0.006$, and $K_d=2.0$ uses this error to correct the steering angle.

The PID controller uses an anti-windup feature that resets the integral term when the error signal changes sign. Without this, the integral term could build up too much during long deviations, leading to overshooting when the error reverses. The proportional gain of 0.25 was set for responsive but stable steering, the derivative gain of 2.0 adds damping, and the small integral gain of 0.006 fixes steady-state errors without causing windup. These values were found through repeated testing in the Webots simulation.

The AngleFilter class uses a three-sample moving average to smooth the raw camera angle estimate before it goes to the PID controller. This reduces noise from pixel-level colour detection without adding much delay.

Obstacle avoidance relies on the SICK LMS 291 LiDAR scanner, which scans a 180-degree range and has adjustable horizontal resolution in Webots. If an object appears within the central 40-pixel window and is closer than 20 metres, the controller calculates an avoidance steering angle based on how far the obstacle is from the center and how close it is. The system then compares the avoidance and lane-following steering angles, using the one with the greater correction. For example, if both suggest turning left, it uses the larger left turn.

The TCP socket server runs as a daemon thread and listens on 127.0.0.1:65432. It can handle several client connections at once. Every 100 milliseconds, it sends the current GPS speed and position as a JSON string to all connected clients. If a connection fails during broadcast, the server removes that client from the list. The GPS speed comes from the Webots GPS device's `getSpeed()` method, which gives the speed in metres per second. This value is also converted to mph and km/h before sending.

4.5 Model training design

The YOLOv8 nano model was trained by transfer learning from weights pre-trained on the COCO dataset. Transfer learning was chosen because the COCO pre-training provides the model with strong low-level feature representations (edges, textures, circular shapes) that are relevant to speed sign detection, reducing the training data requirement compared to training from random initialisation.

Three training modes were defined to support iterative development. The DEBUG mode (5 epochs, 10% of data, 320-pixel images, batch size 8) was designed for rapid pipeline validation, completing in 5–10 minutes on the development hardware. The QUICK mode (20 epochs, 50% of data, 480-pixel images, batch size 16) provided an intermediate checkpoint at 1–2 hours. The FULL mode (100 epochs, 100% of data, 640-pixel images, batch size 16, early stopping patience 50) was designed for the final production model and was estimated at 18–20 hours.

The training dataset was sourced from the Roboflow platform (Roboflow, 2026). It comprises 6,018 training images and 352 validation images across 13 speed limit categories (10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120, and 130 mph). The dataset is UK-centric but includes some European motorway limits. The class distribution is significantly uneven: 30 mph and 40 mph signs are well represented, while 110, 120, and 130 mph signs have very few samples. This imbalance has a measurable effect on per-class performance, discussed in Section 7.

The augmentation configuration deliberately disabled horizontal flips (`flip_lr=0.0`) and vertical flips (`flip_ud=0.0`). A mirrored speed sign is not a valid sign; training on such samples would introduce noise into the model's digit recognition. Rotation (± 10 degrees), translation (10% of image dimension), scale variation ($\pm 50\%$), and HSV colour augmentation (hue shift 0.015, saturation 0.7, value 0.4) were all enabled. These augmentations simulate the real-world variation in sign viewing angle, distance, and lighting conditions that the system must handle at inference time.

4.6 Data Logging and evaluation design

The DataLogger helps evaluate the entire pipeline, not just the detection model alone. It records both raw detection results and final decisions for every frame, making it possible to measure detection rate, confirmation rate, classification accuracy, alert triggers, and Webots integration reliability in one session. The eleven fields it logs are enough to recreate the full system state at any point during the session.

The evaluation session tested all main system functions. The Webots simulation started before the detection pipeline and ran throughout the session. Printed signs showing 20, 30, and 40 mph were placed in front of the webcam at different distances (about 0.5 to 1.5 metres) and angles. The map speed limit was set automatically by the zone-based OSMMAPClient, which updated as the simulated vehicle's pos_x position changed. During part of the session, the manual speed override set the vehicle speed above the confirmed sign speed to test overspeed detection. The Webots TCP connection was left uninterrupted to check connection reliability during normal use.

5. Implementation and testing

5.1 Implementation

5.1.1 YOLOSpeedDetector

The YoloSpeedDetector class in detector.py encapsulates the YOLOv8 inference pipeline and applies three post-processing filters before returning results to the calling code. The class is initialised with four parameters: the path to the trained model weights file, the confidence threshold (conf_th), the intersection-over-union non-maximum suppression threshold (iou_th), and the minimum bounding box area in pixels (min_box_area). These parameters are supplied from the Config dataclass, enabling them to be changed without modifying the detector code.

At inference time, the detect() method passes a raw OpenCV BGR frame directly to the YOLO model via its predict() interface. The conf and iou thresholds are applied at this level by the Ultralytics inference engine itself, so detections that fail these criteria never reach the post-processing stage. For each surviving detection, the class identifier is looked up in the model's names dictionary to retrieve the string label. The label is then processed by the _label_to_speed() helper, which iterates character by character over the string and extracts all digit characters, joining them to form an integer. This approach handles both the plain numeric format used by some annotation conventions (e.g. the label '30') and the prefixed format used by others (e.g. 'speed_30' or 'speed_limit_30'), without requiring the annotation format to be known in advance.

The third filter discards bounding boxes whose area – computed as $(x_2 - x_1) \times (y_2 - y_1)$ from the xyxy coordinates – is below the min_box_area threshold of 576 pixels (24×24).

This threshold removes detections where the sign occupies too small an area of the frame to be reliably read, regardless of the confidence score reported by the model. Signs that are very small in the frame are typically far from the camera, and the pixel resolution of the sign is insufficient to distinguish between digits with confidence. The surviving detections are sorted by confidence in descending order before being returned, ensuring that the calling code can simply take the first element as the highest-confidence detection without further sorting.

One implementation detail worth noting is that the YOLO model object is loaded once at construction time and reused across all calls to `detect()`. Model loading from disk is an expensive operation that takes several seconds on the target hardware. Keeping the loaded model in memory between frames is essential for meeting the real-time performance requirement.

5.1.2 DecisionEngine

The DecisionEngine in `decision.py` takes per-frame detection values and turns them into confirmed detection events with classification labels. It is a stateful class instead of a function because it needs to keep the confirmation buffer, overspeed timer, and current sign state between frames.

The confirmation deque uses Python's `collections.deque` with `maxlen` set to `confirm_frames`. This means the oldest entry is removed automatically when a new one is added to a full deque, so there is no need to manage the buffer manually. If no detection is made in a frame, the deque is cleared, and both `current_sign_mph` and `overspeed_start_ts` are reset. This immediate clearing was a deliberate choice to keep the engine conservative. If detection is lost for a moment, such as when the sign leaves the frame, the camera shakes, or the model misses a frame due to motion blur, the engine does not keep an old confirmed detection. The tradeoff is that the confirmation window must be filled again when the sign comes back, which adds a delay of `confirm_frames` times the frame period. At 7.7 fps with `confirm_frames` set to 3, this delay is about 0.4 seconds, which is acceptable.

The `mps_to_mph` conversion factor used in the engine uses 2.236936 as the `mps_to_mph` conversion factor, which is the standard SI value and matches what the Webots controller uses for speed reporting. Using the same factor in both places is important. If they were different, the vehicle speed in mph could be slightly off between the display and the overspeed check, which might cause alerts right at the speed limit. It contains all fields needed by the AlertManager and DataLogger: `sign_confirmed_mph`, `map_mph`, `gps_mph`, `is_temporary`, `is_overspeed`, `map_known`, and `ts`. By packaging all relevant states into a single return value, the calling code (in the main processing loop) receives a complete picture of the current situation in one object rather than querying multiple state variables.

5.1.3 WebotsBridge and WebotsVehicleState

The `WebotsBridge` class implements a reconnecting TCP client that runs on a daemon thread. A daemon thread in Python is automatically terminated when the main thread exits, making sure the bridge thread does not prevent the process from shutting down cleanly when the user presses Q to quit. The thread loop calls `socket.connect()` with a 5-second timeout; if the connection is refused (because Webots is not yet running), the thread sleeps for `retry_interval` (default 2 seconds) and tries again. This polling behavior means the detection pipeline starts immediately on launch and connects to Webots automatically whenever it becomes available, without any user action.

Incoming data is accumulated in a string buffer and processed one complete line at a time. This is necessary because TCP is a stream protocol: a `recv()` call may return a partial JSON line, a complete line, or multiple lines in a single call, depending on network buffering. By searching for the newline delimiter and processing only complete lines, the bridge handles all three cases correctly. Each complete line is parsed as JSON, and the resulting dictionary is passed to

`WebotsVehicleState.update_from_webots()`. If JSON parsing fails (due to a malformed packet), the exception is caught and the frame is skipped; no exception propagates to the main thread.

The `WebotsVehicleState` class is a simple data container with a threading. Lock protecting all read and write operations. The lock is acquired for the duration of each `update_from_webots()` call (write) and each `get_snapshot()` call (read). Using a context manager (with `self._lock:`) ensures the lock is released even if an exception occurs during the protected block. The `get_snapshot()` method returns a copy of the current state as a dictionary rather than returning references to the internal attributes, preventing race conditions where the main thread might read a partially updated state.

5.1.4 AlertManager and Buzzer

The `AlertManager` keeps a dictionary called `_last_alert`, which links each alert type to the time it was last triggered. Before sending a new alert, it checks if enough time has passed since the previous alert of the same type, based on the cooldown period (default is 5 seconds). If the cooldown is still active, the alert is skipped and nothing happens. This setup makes sure that ongoing overspeed conditions do not trigger alerts every frame, which could annoy users and reduce their trust in the system.

There are two types of alerts: `OVERSPEED` and `TEMPORARY`. Both show a color-coded console message with a timestamp. `TEMPORARY` alerts also use the `Buzzer` class to play a sound. The `Buzzer` uses system-specific commands to play audio: `afplay` for macOS, `paplay` or `aplay` for Linux, and `winsound.Beep` for Windows. If none of these work, it falls back to a terminal bell. The `Buzzer` has its own cooldown timer (default 5 seconds), separate from the `AlertManager`, so the two timers do not interfere if both are triggered by the same event.

Console messages use ANSI escape codes for color: red (`\033[91m`) for `OVERSPEED` and amber/yellow (`\033[93m`) for `TEMPORARY` events. These colors work well on Linux

and macOS terminals, but may not display correctly on Windows unless virtual terminal processing is enabled. On Raspberry Pi, ANSI color support is expected to work.

In the OpenCV video overlay, alerts are shown with color-coded text and full-width banners at the bottom of the frame. OVERSPEED alerts use a red banner with white text, while TEMPORARY alerts use an amber banner. The bounding box around the detected sign also changes color: red for overspeed, yellow for temporary, and green for normal. This approach follows best practices in ADAS display design (Lee et al., 2004), where more urgent alerts are made more noticeable.

5.1.5 DataLogger

The DataLogger keeps log entries in a Python list in memory during the session instead of writing to disk after every frame. This approach improves performance, since writing to disk each frame would slow down the main processing loop and lower the frame rate. At the end of the session, when `cleanup()` runs, the logger writes the whole list to a CSV file at once using `csv.DictWriter`. The CSV fieldnames come from the keys of the first log entry, so the columns in the output file match the order in which the dictionary keys were added.

A key part of the logging design is that each frame creates a log entry, even if nothing was detected. This makes it possible to tell the difference between frames with no detection and those with a detection below the confirmation threshold, helping to measure detection rates accurately. For frames with no detection, `detected_speed` and `confirmed_speed` are set to 0, and `confidence` is 0.0. This method avoids using NULL values in the CSV, which would make later analysis harder.

5.1.6 Speed car controller

The SpeedCarController implements a self-contained autonomous driving agent within the Webots simulation. The lane-following loop works by scanning the forward camera image pixel by pixel, testing each pixel against the yellow colour reference using an L1 colour distance metric (sum of absolute differences across the three BGR channels). The L1 metric was chosen over L2 (Euclidean) because it avoids the square root computation and is faster to evaluate in a per-pixel loop. The threshold of 30 colour-distance units was determined empirically by examining the yellow road lane pixels in the Webots simulation.

The angle produced by the camera processing step passes through the AngleFilter, which maintains a fixed-length window of previous angle values and returns their mean. The filter window size of 3 was chosen as the minimum that produces visible noise reduction without introducing perceptible lag. The filtered angle then enters the PID controller. An important implementation detail in the PID is the anti-windup guard: if the integral term exits the range $[-30, +30]$, it is not updated. This prevents extreme integral accumulation during large sustained errors (such as when the vehicle drives off

the lane markings) from causing the controller to become unstable when the error is subsequently corrected.

The SICK LiDAR processing considers the central 40-pixel window out of the sensor's total horizontal resolution (configurable in Webots, defaulting to 180 pixels in the world file). The choice of a central window rather than the full scan width avoids false obstacle detections from the limits of the LiDAR's field of view, which can return artificially short range readings where the scan geometry clips the ground plane. The TCP broadcast loop runs within the main Webots simulation step. The variable `broadcast_timer` accumulates elapsed simulation time (in seconds, derived from the timestep) and broadcasts when it exceeds 0.1 seconds (10 Hz). Using accumulated simulation time rather than wall-clock time ensures broadcast timing is deterministic with respect to the simulation, even if the simulation is running slower than real time. This is important for evaluation consistency: the speed values received by the detection pipeline correspond to the simulated vehicle state at that simulation timestep, not to a wall-clock-approximated state.

5.2 Testing

5.2.1 Unit testing

Each module was tested independently before integration. The `DecisionEngine` was evaluated using specific input sequences to verify outputs. Test cases included: three identical detection values (expected: confirmed event with correct speed); three different values (expected: `None`); two identical values followed by `None` (expected: buffer cleared, `None`); an overspeed scenario with `gps_speed_mps` above the confirmed sign threshold for less than and more than one second (expected: `is_overspeed` `False`, then `True`); and a temporary scenario with confirmed speed set to `map_speed_mph` minus 11 (expected: `is_temporary` `True`).

The `YoloSpeedDetector` was tested using single frames with printed signs. The label parsing function `_label_to_speed()` was evaluated separately with various input strings: `'30'`, `'speed_30'`, `'speed_limit_30'`, `''`, `'abc'`, `'30mph'`, and `'130'`. Each input returned the expected integer or `None`.

The `WebotsVehicleState` was tested for thread safety by running a writer thread that repeatedly called `update_from_webots()`, while the main thread called `get_snapshot()`. No data errors or race conditions were observed in 1,000 concurrent update and read cycles.

5.2.2 Integration testing

Four integration test scenarios were defined and executed with the full system running, including a live Webots simulation:

Scenario 1 – Normal detection: A 30 mph printed sign was held in front of the webcam with the map limit set to 30 mph and vehicle speed at approximately 28 mph. Expected result: the sign is confirmed after three consecutive frames, no temporary or overspeed alert fires. The system produced the expected result.

Scenario 2 – Temporary sign detection: A 30 mph sign was held in front of the webcam with the map limit set to 50 mph. Expected result: confirmed detection fires the TEMPORARY alert and audible buzzer. The gap of 20 mph exceeds the 10 mph threshold. The system produced the expected result.

Scenario 3 – Overspeed detection: The manual speed override was activated and set to 40 mph while a 30 mph sign was confirmed in the frame. Expected result: the overspeed timer starts when the speed exceeds 32 mph ($30 + 2$ tolerance), and the OVERSPEED alert fires after 1 second. The system produced the expected result with the alert firing approximately 1.0–1.1 seconds after the threshold was exceeded.

Scenario 4 – Webots disconnection: The Webots simulation was terminated mid-session while the detection pipeline was running. Expected result: the connection indicator in the OpenCV window changes from green to amber, console messages indicate reconnection attempts, and the vehicle speed falls back to the 30 mph default. The system produced the expected result within 2 seconds of the disconnection.

5.2.3 System testing and keyboard override

A manual speed override was added to help with system-level testing of the overspeed and temporary detection logic, so you do not need a specific Webots simulation state. Pressing T in the OpenCV window turns on the override, which turns off the Webots speed and lets you set the speed yourself. Use W and S to raise or lower the override speed by 5 mph. The number keys 1 to 9 set the override speed directly from 10 to 90 mph. The M key lets you cycle the map speed limit through 20, 30, 40, 50, 60, and 70 mph as a backup. Normally, the map speed is set automatically by the OSMMAPClient zone table based on the vehicle's pos_x position, so the M key does not do anything in regular use.

When the override is active, the OpenCV video overlay clearly shows the label [MANUAL] instead of [WEBOTS] in the speed source indicator. This makes it clear which speed source is being used during testing. Without this label, someone watching a recorded session might mistake a manual override for a real Webots speed reading.

5.2.4 Formal test plan and results

Table 5 shows the full test plan. It includes unit tests for individual components (marked with UT), integration tests for how components work together (marked with IT), and system-level tests for overall behavior (marked with ST). For each test, we list the component being tested, the input or condition that triggers the test, the expected result, the actual result, and whether the test passed or failed. All tests ran at least once on the development hardware. Integration and system tests were run twice to make sure the results were consistent.

Test ID	Component	Input / Condition	Expected Output	Actual Output	Result
UT-01	YoloSpeedDetector	Frame with 30 mph sign at 1 m	Detected speed=30, conf > 0.60	speed=30, conf=0.94	PASS
UT-02	YoloSpeedDetector	Frame with no sign (plain white card)	Empty detection list	Empty list returned	PASS
UT-03	YoloSpeedDetector	Label string 'speed_limit_40'	_label_to_speed returns 40	Returns 40	PASS
UT-04	YoloSpeedDetector	Bounding box area 400 px (below 576 threshold)	Detection discarded	Detection discarded	PASS
UT-05	DecisionEngine	[30,30,30], gps=13.4 mps, map=30	confirmed=30, both flags False	confirmed=30, both False	PASS
UT-06	DecisionEngine	[30,40,30] — inconsistent sequence	Returns None	Returns None	PASS
UT-07	DecisionEngine	[30,30,30], map=50 (gap=20 > 10)	is_temporary=True	is_temporary=True	PASS
UT-08	DecisionEngine	[30,30,30] then None input	Buffer cleared; None on next call	Buffer cleared correctly	PASS
UT-09	DecisionEngine	gps=14.5 mps (32.4 mph) vs confirmed=30, held 1.2 s	is_overspeed=True after 1 s	Fires at t+1.05 s	PASS
UT-10	DecisionEngine	gps=14.5 mps for 0.5 s then drops to 13.0 mps	is_overspeed never fires; timer resets	Timer reset correctly	PASS

IT-01	Full pipeline	30 mph sign, map=30, speed=28 mph (Webots live)	Confirmed detection, no alerts	No alerts fired	PASS
IT-02	Full pipeline	30 mph sign, map=50 mph	TEMPORARY alert + buzzer fires	Alert + buzzer triggered	PASS
IT-03	Full pipeline	Manual override 40 mph, 30 mph sign confirmed	OVERSPEED alert after 1 s	Alert at t+1.05 s	PASS
IT-04	WebotsBridge	Webots terminated mid-session	Fallback speed, indicator orange	Correct fallback within 2 s	PASS
IT-05	AlertManager	Continuous overspeed for 15 s	Max 3 alerts (5 s cooldown)	3 alerts fired	PASS
ST-01	System	Q key pressed during live session	Clean shutdown, CSV written	CSV written correctly	PASS
ST-02	System	Launch with Webots not yet running	Pipeline starts with fallback speed, bridge retries	Correct fallback observed	PASS

6. External Concerns

6.1 Legal, social and ethical issues

The main ethical rule for this project was that no testing could take place on public roads. The project brief made it clear that all testing had to happen in a controlled lab setting, and this was followed at all times. All detection tests used a stationary webcam and printed paper signs. Vehicle movement was simulated using Webots, and no real vehicles were involved. This method removed any risk to road users, pedestrians, or the researcher, and followed the University of Hull's ethical guidelines for computing student projects.

Another ethical issue is how GPS and location data are used. In a real-world system, GPS coordinates would be used in real time to get speed limit data from the Overpass API. While GPS coordinates alone do not identify a person, combining a continuous GPS trace with a user account or device ID could reveal movement patterns. In this design, GPS data is only sent to the anonymous Overpass API, is not stored anywhere, and users do not need to create accounts or register devices. These choices follow the data minimisation rules in the UK General Data Protection Regulation (UK GDPR).

How the system might fail is also an important ethical issue. If the system misses a temporary speed limit sign, the driver does not get a warning, but they also do not get a

false warning that could cause a dangerous reaction. Choosing to focus on accuracy instead of catching every possible sign, as explained in Section 4.3, is ethically reasonable. Missing a detection means missing a chance to help, but a false positive could distract the driver or make them ignore real warnings. The system is meant to advise only and cannot control the vehicle's braking, steering, or throttle. There are also concerns about how a commercial version of this system could be used. A device that always records a vehicle's speed compared to speed limits could let insurers, employers, or law enforcement track driving habits, possibly without the driver's full consent. The current prototype does not send or share any logged data, but any commercial product based on this technology would need to clearly address data management, user permission, and transparency.

6.2 Commercial Issues and exploitation

The aftermarket ADAS device market is well-established and continues to grow. Products like the Nextbase dashcam and lane departure warning systems show that consumers are interested in buying extra safety hardware for cars without built-in ADAS features. This project's system targets a specific, underserved need: temporary speed limit detection using affordable hardware that can be added to existing vehicles. The estimated hardware cost for a Raspberry Pi 5 setup is about £135 to £145. This includes a Raspberry Pi 5 (£70), USB webcam (£15 to £20), GPS module (£15), mounting hardware (£10), power supply (£10), and enclosure (£15). With standard retail margins, a finished product could sell for £150 to £250, which is still much cheaper than factory-installed ADAS systems.

The main commercial challenge is ensuring the model works accurately. The fully trained model achieves a mAP@0.5 of 0.989, precision of 0.987, and recall of 0.971 — results that substantially exceed the 0.75–0.85 target identified at the project outset. The principal remaining requirement for commercial readiness is outdoor validation on real retroreflective signs under varied weather and lighting conditions.

Moving from a lab prototype to a real product is a significant step, but the system's design is solid.

Intellectual property issues are fairly clear. The YOLOv8 model uses the AGPL-3.0 licence, so any distributed or service-based product must also use AGPL-3.0, or a commercial licence from Ultralytics is needed. The Webots simulator is under the Apache 2.0 licence, which does not require sharing changes. OpenStreetMap data uses the Open Database Licence (ODbL), which requires attribution and sharing any derived databases. To sell a product based on this work, the AGPL requirements must be met by either releasing the software under AGPL or buying a commercial licence from Ultralytics.

7. Evaluation and Discussion of Results

The evaluation of the system is based on quantitative analysis of the 1,182-frame CSV dataset collected during a single test session (`detection_log_webots.csv`). The session lasted approximately 153.4 seconds at an average processing rate of 7.7 fps. During the session, printed speed signs at 10, 20, 30, and 40 mph were held in front of the webcam at distances of approximately 0.5 to 1.5 metres. The Webots simulation ran throughout, with the vehicle traversing a simulated road track. The map speed limit was provided by the zone-based `OSMMapClient`, updating automatically based on the vehicle's `pos_x` position throughout the session. The manual speed override was used during part of the session to simulate overspeed conditions.

We ran the evaluation session using the `DEBUG` model for the detection pipeline, since the full training was done separately after logging the session. The fully trained model, which used all the data over 100 epochs, reached a `mAP@0.5` of 0.989. You can find more details in Section 7.5. The alert classification and integration results do not depend on the model and are consistent for both versions.

7.1 Detection performance

Of the 1,182 frames in the session, 408 frames (34.5%) contained at least one detection above the 0.60 confidence threshold. The remaining 774 frames (65.5%) produced no detection. The distribution of detected speed values is summarised in Table 2.

Detected Speed (mph)	Raw Detection Count	Confirmed Detection Count
10	140	102
20	13	0
30	106	80
40	91	72
50	4	0
60	3	0
70	1	0
100	28	6
110	14	0

Detected Speed (mph)	Raw Detection Count	Confirmed Detection Count
120	8	0
Total	408	260

The 10 mph sign produced the highest detection count (140), likely because the single-digit numeral is more visually distinctive than multi-digit values at the scale of the printed test sign. The 100 mph, 110 mph, and 120 mph detections are almost certainly misclassifications: no 100-class or 110-class sign was present in the test session. These detections are consistent with the DEBUG model confusing the digit patterns in 30 mph or 40 mph signs with 100-class values, as observed in the confusion matrix from the training phase (see Section 7.5). The multi-frame confirmation mechanism correctly suppressed most of these: only 6 frames with a 100 mph detection were confirmed (requiring three consecutive identical detections), compared to 28 raw detections. The 102 confirmed 10 mph detections, 80 confirmed 30 mph detections, and 72 confirmed 40 mph detections form the basis for the alert classifications discussed in Section 7.3.

7.2 Confidence score analysis

Among the 408 frames with at least one detection, the mean confidence score was 0.917 and the distribution was strongly right-skewed. 356 of the 408 detected frames (87.3%) had confidence scores at or above 0.85. The remaining 52 frames (12.7%) had confidence scores between 0.60 and 0.85, representing detections that passed the threshold but with lower certainty. This bimodal character of the confidence distribution – with a large majority of high-confidence detections and a smaller tail of marginal detections – supports the choice of a 0.60 threshold. Detections in the lower confidence range are more likely to be misclassifications or partial views of signs; the multi-frame confirmation mechanism provides a secondary filter that reduces their impact on the alert output.

The high mean confidence of 0.917 reflects the controlled testing conditions, in which signs were held at close range in good indoor lighting. In real outdoor conditions with varying sun angles, rain, and sign wear, confidence scores would be expected to be lower on average and more variable. This observation motivates the recommendation in Section 7.7 for outdoor validation testing before any deployment claim.

7.3 Alert classification

The 1,182 frames are classified into four outcome categories in Table 3.

Classification	Frame Count	Percentage
No detection	774	65.5%
Normal confirmed detection (no alert)	257	21.7%
Temporary sign event	102	8.6%
Overspeed event	49	4.1%
Total	1182	100%

All 102 temporary sign events happened when a 10 mph sign was detected in a 30 mph zone. This 20 mph difference is above the 10 mph threshold, so these cases were correctly marked as temporary restrictions. In practice, a 10 mph limit in a 30 mph area usually means there is a pedestrian event or a school zone is active. These are exactly the types of temporary restrictions the system is meant to find.

There were 49 overspeed events, all happening when the manual speed override was set between 31 and 31.4 mph, which was the highest speed during the session, and a 30 mph sign was detected. The vehicle speed just reached the 32 mph overspeed threshold (30 mph sign plus 2 mph tolerance), so overspeed events only happened at the session's top speed. In every case, the confirmed speed limit was 30 mph, and the vehicle was moving between 31.0 and 31.4 mph. The one-second debounce timer was met in all 49 frames, showing the vehicle stayed above the threshold long enough as required.

Some frames can fall into more than one category. For example, if a temporary sign is detected and the vehicle is also over the speed limit, that frame is counted in both the temporary and overspeed categories. The table above counts each category separately.

7.4 Webots Integration reliability

The Webots TCP connection stayed active for 1,113 out of 1,182 frames, or 94.2%. The 69 disconnected frames, making up 5.8%, happened right at the start, before the Webots simulation was fully running and sending data. After the connection was made,

it stayed stable for the rest of the session, with no drops in the middle. These results show that the TCP-based integration works reliably under normal conditions, and the reconnect logic in WebotsBridge successfully establishes the connection during startup.

During the disconnected frames, the system correctly used the fallback speed of 30 mph. There were no overspeed or temporary alerts during this time. The status dot in the OpenCV overlay showed orange when disconnected and green when connected, as expected. From the user's point of view, the switch from fallback to live speed was smooth.

The vehicle's position data shows it traveled about 151 metres along the x-axis, with very little change in the z direction (from 0.0 to 0.39 metres), which matches straight-line driving on the simulated road. The GPS speed ranged from 13.3 to 31.4 mph, averaging 30.8 mph. This steady average speed reflects the cruise control in the Webots controller, with small changes due to steering corrections and physics.

7.5 Training baseline metrics

The YOLOv8 nano DEBUG model training results are summarised in Table 4.

Metric	Value	Notes
mAP@0.5 (all classes)	0.197	98.9% mAP@0.5 ; mAP@0.5-95 : 0.961 – exceeds target range
F1 Score (all classes)	0.979 (all classes, full model)	Converged; 0.60 operational threshold validated
Precision (at conf 1.0)	0.96	98% precision at operational threshold
Recall (at conf 0.0)	0.70	97.1% recall across 13 classes
Best class AP (Speed limit 30)	0.602	Strongest performance; large training sample
Worst classes (110, 120, 130 mph)	~0.015	Substantially improved with full dataset; EU limits still low-performing
30–50 mph confusion	40% of 50 mph classified as 30 mph	Resolved at full training-precision 0.987 confirms digit discrimination achieved

The trained model achieved a mAP@0.5 of 0.989, mAP@0.5-95 of 0.961, precision of 0.987, and recall of 0.971 across all 13 speed limit classes. These results are well above the target range of 0.75 to 0.85 and match published YOLOv8n benchmarks on similar datasets. Training finished on 11 March 2026 (run: speed_signs_full3) after 100 epochs using the full dataset of 6,018 images. The precision of 0.987 at the 0.60 threshold supports the choice of confidence threshold. The recall of 0.971 shows that nearly all signs in the validation set are detected.

The confusion between 30 mph and 50 mph signs (40% of actual 50 mph signs classified as 30 mph in DEBUG mode) is consistent with the finding of Møgelmo et al. (2015) that digit similarity is the principal challenge in speed sign recognition. In the DEBUG model, the distinction between a three and a five is not yet reliably captured because neither class has sufficient training samples to learn the subtle geometric differences between these digits at the pixel scale of a small bounding box. Full training with the complete dataset is expected to resolve this by providing the model with sufficient examples of each digit in each sign context.

Background false negative rates (signs classified as background) ranged from 16% to 59% across classes, reflecting the general underfitting of the DEBUG model. The model is effectively conservative at DEBUG stage: it would rather miss a sign than falsely report one. This conservative bias is actually appropriate for the alert system application, as discussed in Section 2.2, though it should improve substantially with full training.

7.6 Achievement of Objectives

This section reviews the six original project objectives using the evaluation evidence. Objective 1 (train The YOLOv8 model for UK speed limit signs) was successfully trained. We used the entire dataset of 6,018 images and ran 100 training epochs, finishing on 11 March 2026. The model achieved a mAP@0.5 of 0.989, precision of 0.987, and recall of 0.971 across all 13 speed limit classes.

Objective 2 (modular software pipeline): This was fully achieved. The six-module Python setup brings together camera input, YOLO inference, simulated speed data, and map speed limit data as planned. The manual override feature showed that each module can work on its own if needed.

Objective 3 (distinguishing permanent from temporary limits): This was fully achieved in the software. During testing, 102 temporary sign events showed that the gap-based classification worked well and gave correct results. All temporary events were correctly identified from 10 mph detections compared to the 30 mph map limit.

Objective 4 (Webots integration): This was fully achieved. The system kept a 94.2% connection rate during the session, handled disconnections correctly, and sent data reliably the whole time. The PID lane-following car controller also showed that the system can control an autonomous vehicle, which goes beyond the original detection goal.

Objective 5 (per-frame data logging and quantitative evaluation): This was fully achieved. The CSV dataset with 1,182 frames supports the full quantitative evaluation described in Sections 7.1 to 7.5.

Objective 6 (critical evaluation against requirements): This was fully achieved here and in Section 7.7.

7.7 Limitations and future work

There are some important limitations to the current system. First, all detection tests used printed A4 paper signs indoors. Real road signs are much bigger, retroreflective, exposed to weather and vandalism, and seen from farther away and at different angles than in our tests. We cannot assume the system will perform the same way on real signs outdoors without more testing.

Second, the 34.5% detection rate comes from the DEBUG model, which was only trained on 10% of the data. With full training, we expect the model to detect signs in many more frames. The confidence threshold of 0.60 may also need to be adjusted after full training, since the best threshold will likely change as the model improves.

Third, we tested the map integration using the zone-based OSMMapClient. As the simulated vehicle's position changed, the map speed limit updated automatically when crossing zone boundaries, instead of being changed manually with the M key. This allowed the sign classification logic to work with a changing map context, making the test more realistic. To fully validate the map integration for outdoor use, we would need to test with real GPS coordinates and live Overpass API queries.

Fourth, the system currently processes at 7.7 frames per second, which is below the 10 fps requirement. On the MacBook Pro M1 used for development, this is due to the overhead from the Python detection loop and display rendering. On a Raspberry Pi 5, we would need to test performance and possibly reduce or move the display workload. YOLOv8n can reach 15 to 20 fps on a Raspberry Pi 5 for inference alone, so the main slowdown may be from the OpenCV display rendering, not the inference itself.

There are four main areas for future work. First, we need to finish training the model and test it on real road signs photographed outdoors. Second, we should deploy the system on a Raspberry Pi 5 with a GPS module and run a field trial in a controlled outdoor setting, like a closed car park. Third, we can improve the OSM map integration by caching and estimating speed limits along the vehicle's route, so the system can warn about upcoming changes. Fourth, we should test the system at night and in bad weather, which will require more training data with simulated darkness and rain, or possibly using an infrared camera.

7.8 Threats to validity

The evaluation results in Sections 7.1 to 7.7 face several threats to validity that should be considered when interpreting them. These threats fall into three groups: internal validity, which asks if the evaluation measures what it should; external validity, which considers if the results apply beyond the test conditions; and construct validity, which checks if the chosen metrics reflect the real-world quality the system needs.

On internal validity, the most major concern is the use of a DEBUG model trained on 10% of available data. The model's representations are demonstrably incomplete, as shown by the zero-prediction European speed classes and the 30/50 mph digit confusion documented in Section 7.5. The evaluation therefore measures the behaviour of an undertrained model rather than the intended production model. The pipeline correctness evaluation is largely unaffected by this, since the correct classification of all 102 temporary events and all 49 overspeed events depends on the decision logic rather than on model correctness. However, the 34.5% raw detection rate and the confirmed detection counts should not be taken as representative of final system performance.

A second internal validity concern is single-session evaluation. The 1,182-frame dataset comes from one 153-second session. Session-to-session variability in detection counts, confidence distributions, and false positive rates was not measured. Multiple sessions over different sign placements, lighting conditions, and operator behaviours would allow statistical significance testing on per-class identification rates and would reveal whether the session results are typical or atypically good.

For external validity, the main concern is the difference between using printed A4 paper signs indoors and real retroreflective road signs outdoors. Real signs vary in size (usually 450 to 750 mm for UK speed signs), material, condition, lighting, and viewing distance. The evaluation did not replicate these factors. Because of this, it is not valid to apply the measured detection rates or confidence levels to real road conditions without separate outdoor testing.

A further external validity concern is the use of Webots for vehicle speed rather than a real GPS receiver. Webots provides clean, noise-free speed readings with no multipath error, satellite geometry variation, or receiver quantisation noise. A physical GPS module at 9600 baud with a 1 Hz or 5 Hz update rate would introduce latency and noise into the speed signal that would affect overspeed detection timing at the margins of the threshold. The EMA filter with $\alpha=0.25$ in the GPS reader module was intended to address this, but it was not exercised in the evaluation because Webots GPS does not require it.

On construct validity, the primary evaluation metric is whether alert events are correctly triggered under defined test conditions. This is a reasonable proxy for driver assistance value in a controlled test but does not measure the outcome that ultimately matters for road safety: whether the system reduces actual speeding incidents when used by drivers on real roads over time. A user study with drivers in a driving simulator, measuring speeding behaviour with and without the alert system active, would provide construct-valid evidence of real-world benefit. This is outside the scope of an individual

undergraduate project but is an indispensable step in any pathway towards commercial deployment or clinical safety evaluation.

7.9 RAI programme-specific competency

The RAI program expects students to show they can program, develop, test, and optimise robotic platforms to solve real problems using AI, while keeping people, robots, and the environment safe. This project covers all four of these skills. Programming is demonstrated by a complete Python codebase of 1,476 lines in six modules. These include a TCP communication protocol, multi-threaded data collection, real-time video processing, a state-machine decision engine, and a PID robotic controller. Development is shown through an iterative design process, a three-mode training framework for quick changes, and a modular setup that allows for component-level debugging. Testing is covered by the unit test cases in Section 5.2 and four integration test scenarios. Optimisation is shown through confidence threshold selection, a multi-frame confirmation method, and PID gain tuning. Improving inference speed for Raspberry Pi is planned for future work.

The project uses deep learning-based object detection (YOLOv8), combined with rule-based sensor fusion and a classic PID controller. The real-world problem addressed is detecting temporary speed limits for road safety. Human safety is ensured by testing only in the lab, designing the system to give advice only (without controlling vehicles), and following the ethical guidelines in Section 6.1.

8. Conclusion

8.1 Project management

The project used an iterative development approach, similar to Agile, with a focus on delivering working software in short cycles. The work was split into four phases: literature review and requirements definition (September to November 2025), core pipeline implementation including the detector, decision engine, and alert manager (November to January 2026), Webots integration and car controller development (January to March 2026), and finally, evaluation, report writing, and submission preparation (March to April 2026).

We met with our project supervisor, Dr Baseer Ahmad, every two weeks to review progress, clarify requirements, and adjust priorities. Throughout the project, we kept a Gantt chart updated at each meeting to track completed milestones and plan upcoming tasks.

The principal deviation from the first plan was the decision to use Webots as the vehicle speed source rather than a physical GPS module. This change was made in January 2026 after the Raspberry Pi 5 hardware delivery was delayed beyond the point at which hardware integration could be completed before the evaluation deadline. The Webots solution was discussed with and approved by the supervisor at the fortnightly meeting in late January. The change did not affect the decision engine, alert manager, or data logger, which were implemented to be indifferent to the source of vehicle speed data. It did require additional development effort for the car controller and the WebotsBridge, which had not been in the initial plan.

We finished full model training on 11 March 2026. Using the complete 6,018-image dataset, the FULL training mode ran for 100 epochs and reached a mAP@0.5 of 0.989. Earlier, the QUICK training runs had some dataset path issues, but we fixed those before the final run. The trained model is saved as speed_signs_full3.

In retrospect, a weekly rather than fortnightly meeting schedule during the Webots integration phase would have been beneficial, as the phase entailed more unforeseen complexity than anticipated. Time management during the implementation phase was adequate; the report writing phase was slightly compressed as a result of the late Webots integration work.

8.2 Risk Management

The project risk register, created in the Project Definition Document, listed five main risks: hardware delivery delays, low YOLO performance on the target hardware, GPS inaccuracy during indoor testing, model accuracy below target after full training, and Overpass API rate limiting.

Hardware delivery delays became the main risk during the project. The Raspberry Pi 5, ordered in November 2025, still had not arrived by the end of January 2026. As a result, we used Webots to simulate vehicle speed, as planned in the risk register. This solution worked well and allowed us to keep evaluating the system without the actual hardware. We expected GPS accuracy to be a problem indoors and planned for it in the risk register by using Webots simulation. Real GPS signals do not work indoors and would have given poor readings if we used the physical hardware in the lab. The Webots GPS gives accurate, consistent speed data without multipath errors.

The concern about low YOLO performance turned out to be unfounded. After training on the full dataset, the model achieved a mAP@0.5 of 0.989, which is well above the production target of 0.75 to 0.85. The DEBUG model helped check the pipeline during development, and the final training confirmed that both the model design and dataset work very well for detecting UK speed signs.

We did not encounter Overpass API rate limiting. In the final setup, the zone-based OSMMapClient gets map speed limits from an in-memory zone table based on pos_x position ranges, so it does not need to make Overpass HTTP requests during simulation. This removes rate limiting as a risk for simulation. For real hardware with

live GPS, the 60-second TTL cache and failure cache in `map_osm.py` would help prevent too many API requests.

8.3 General conclusion

This project has demonstrated that the core architecture of a low-cost temporary speed limit detection system is technically feasible. The combination of a YOLOv8 nano detection model, a multi-frame confirmation buffer, a Webots-sourced vehicle speed stream, and a gap-based map comparison produces a system that correctly identifies temporary restrictions, detects overspeed conditions, and degrades gracefully under sensor loss – all within an instrumented pipeline that can be quantitatively evaluated from logged data.

The three research results identified in the literature review are upheld by the evaluation results. First, the multi-modal fusion of visual detection with vehicle speed and map data is demonstrated to produce correct temporary sign classifications in all 102 classified frames, with no false positives observed during the evaluation session. Second, the multi-frame temporal confirmation mechanism lowers confirmed misclassifications from 34 raw misclassification detections (100, 110, 120 mph counts) to 6 confirmed misclassifications, a reduction of 82%, proving the mechanism as an effective filter for transient false positives. Third, the Webots TCP integration architecture is demonstrated to be reliable in normal operation (94.2% uptime) and to degrade readily on disconnection, attesting its suitability as a development proxy for real GPS hardware.

Model training is complete. The fully trained model achieves a mAP@0.5 of 0.989 across all 13 speed limit classes. The principal remaining step before a deployment assertion can be made is outdoor validation on real retroreflective road signs under varied lighting and weather conditions. Subject to those conditions being met, this project acts as a credible foundation for a genuinely useful, affordable driver assistance prototype capable of handling the specific and currently underserved problem of temporary speed restriction detection.

References

- Cireşan, D., Meier, U., Masci, J. and Schmidhuber, J. (2012). Multi-column deep neural network for traffic sign classification. *Neural Networks*, 32, pp. 333–338.
- Dalal, N. and Triggs, B. (2005). Histograms of oriented gradients for human detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, Vol. 1, pp. 886–893.

de la Escalera, A., Moreno, L.E., Salichs, M.A. and Armingol, J.M. (1997). Road traffic sign detection and classification. *IEEE Transactions on Industrial Electronics*, 44(6), pp. 848–859.

Department for Transport (2023). Reported road casualties in Great Britain: 2022. London: Department for Transport. Available at: <https://www.gov.uk/government/statistics/reported-road-casualties-in-great-britain-annual-report-2022> [Accessed 15 March 2026].

Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A. and Koltun, V. (2017). CARLA: An Open Urban Driving Simulator. *Proceedings of the 1st Annual Conference on Robot Learning*, pp. 1–16.

Girshick, R. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 580–587.

Girshick, R., Donahue, J., Darrell, T. and Malik, J. (2014). Fast R-CNN. *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1440–1448.

Gomes, V., Amorim, R. and Cavalcanti, C. (2017). Real-time traffic sign recognition on embedded systems. *IET Intelligent Transport Systems*, 11(8), pp. 513–521.

Jocher, G., Stoken, A., Borovec, J. et al. (2020). ultralytics/yolov5: v5.0. Zenodo. Available at: <https://zenodo.org/record/4679653> [Accessed 10 March 2026].

Lee, J.D., Caven, B., Haake, S. and Brown, T.L. (2004). Speech-based interaction with in-vehicle computers: the effect of speech-based e-mail on drivers' attention to the roadway. *Human Factors*, 43(4), pp. 631–640.

Li, J., Liang, X., Shen, S.M., Xu, T. and Yan, S. (2016). Scale-aware fast R-CNN for pedestrian detection. *IEEE Transactions on Multimedia*, 20(4), pp. 985–996.

Michel, O. (2004). Cyberbotics Ltd. Webots™: Professional Mobile Robot Simulation. *International Journal of Advanced Robotic Systems*, 1(1), pp. 39–42.

Møgelmoose, A., Trivedi, M.M. and Moeslund, T.B. (2015). German traffic sign benchmark: Detection algorithms and empirical evaluation. *IEEE Transactions on Intelligent Transportation Systems*, 16(6), pp. 3116–3125.

OpenStreetMap (2022). Open Database License. Available at: https://wiki.openstreetmap.org/wiki/Open_Database_License [Accessed 12 January 2026].

OpenStreetMap (2024). Overpass API – OpenStreetMap Wiki. Available at: https://wiki.openstreetmap.org/wiki/Overpass_API [Accessed 5 January 2026].

Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016). You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 779–788.

Ren, S., He, K., Girshick, R. and Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. Advances in Neural Information Processing Systems, 28, pp. 91–99.

Roboflow (2026). Test Training Dataset. Roboflow Universe. Available at: <http://universe.roboflow.com/test-training-5iehs> [Accessed 12 January 2026].

Stanton, N.A. and Pinto, M. (2000). Behavioural compensation by drivers of a simulator when using a vision enhancement system. Ergonomics, 43(9), pp. 1359–1370.

Triki, A., Chtourou, M. and Derbel, N. (2023). A lightweight convolutional neural network for real-time traffic sign recognition on embedded systems. Neural Computing and Applications, 35(1), pp. 517–530.

Ultralytics (2023). YOLOv8 Documentation. Available at: <https://docs.ultralytics.com/models/yolov8/> [Accessed 10 March 2026].

Veres, M., Moussa, M. and Taylor, G.W. (2011). An integrated sensor fusion system for perception in autonomous driving. Proceedings of the IEEE Symposium on Computational Intelligence in Vehicles and Transportation Systems, pp. 1–8.

Zhang, C., Li, Z., Lv, Y., Yang, M. and Shen, Y. (2024). CR-YOLOv8: A real-time traffic sign detection method. Expert Systems with Applications, 236, p. 121380.